Break the Login

Atacarea și Securizarea Autentificării

Proiect #2

Dezvoltarea Aplicațiilor Software Securizate

Facultatea de Matematică și Informatică
Universitatea din București
Coordonator:
Conf. Univ. Dr. Marius Iulian Mihăilescu
marius-iulian.mihailescu@unibuc.ro
Student:
Robert Takacs
robert.takacs.503@gmail.com

Cuprins

Cuprins 2
Introducere 3
Arhitectură 3
Setup Mediu 4
Instalarea aplicației 4
Instalarea mașinii virtuale 4
Implementare MVP 5
Arhitectură pre-existentă 5
Autentificare 5
Audit 5
Tichete 5
Autorizare 6
Vulnerabilități 7
Stocarea parolelor 7
Cerințe de securitate ale parolelor 11
Resetarea Parolelor 16
Operații Neautorizate 22
Brute-forcing 26
Enumerare de conturi 28
Concluzii 34
Link-uri 35
Git Repository: 35
Video-uri: 35

Introducere

În cadrul proiectului „Break the Login”, implementăm o aplicație web destinată companiei fictive „AuthX”, folosită de angajați pentru acces la resurse sensibile și pentru crearea și rezolvarea tichetelor.
Proiectul presupune construirea și mai apoi analiza unei versiuni nesecurizate a aplicației, cu probleme cum ar fi stocare nesigură a parolelor, riscuri la resetarea contului sau erori de autorizare, urmate de prezentarea rezolvării acestor probleme.

Arhitectură

Proiectul este dezvoltat utilizând o arhitectură standard de dezvoltare web. Serverul HTTP este scris cu ajutorul unui framework custom de tip Model-View-Controller creat in-house, utilizând limbajul *PHP 8.3*. Pentru design, se folosește *CSS* cu preprocesorul *SASS*, iar pentru interactivitate front-end, *Javascript jQuery*. Baza de date aleasă este MySQL, și este accesată utilizând MySQL Workbench, iar pentru a simula un mailserver SMTP, se folosește *Mailpit*, toate aceste servicii fiind împachetate în containere utilizând *Docker Compose*. Pentru versionare, se folosește *git*.

Scop	Unealtă
Webserver	Apache
Limbaj server	PHP 8.3
Design	CSS + SASS
Interactivitate	Javascript + jQuery
MVC	MVC Custom
Bază de date	MySQL
Acces bază de date	MySQL Workbench
Mailserver (Mock)	Mailpit
Containerizare	Docker Compose
Platformă Mașini Virtuale	VirtualBox
Mașină Virtuală	Kali Linux (Latest)
Host	Ubuntu 24.04

Tabel 1. Descrierea arhitecturii

Setup Mediu

Instalarea aplicației

Instalarea aplicației se face simplu, utilizând docker-compose și urmând instrucțiunile din fișierul README.md al repository-ului git.

Installation

```
# create config
cp config.php.example config.php

# run sass parsing
sass -w ./

# start containers
docker compose up --build
```

Astfel, se copiază exemplul fișierului de configurație al aplicației în fișierul real de configurație, se pornește SASS pentru a construi fișierele CSS, după care se containerizează aplicația utilizând docker-compose.

Instalarea mașinii virtuale

Mașina virtuală utilizată pentru demonstrațiile acestei aplicații este instalată cu ajutorul VirtualBox pe hostul Ubuntu 24. https://www.virtualbox.org/wiki/Linux_Downloads

Mai apoi, Kali Linux este instalat https://www.kali.org/get-kali/#kali-virtual-machines utilizând installer-ul pentru VirtualBox. Mașina virtuală este pornită și poate fi utilizată imediat.

Implementare MVP

Arhitectură pre-existentă

MVC-ul folosit pentru acest proiect prezintă anumite funcționalități preimplementate, care au existat la momentul inițierii proiectului. Un fișier PHP creat într-un subfolder al folderului `controllers` poate fi accesat prin URL în browser fără alte setup-uri extra. De asemenea, fișierele din folderul `system` sunt preîncărcate la rularea oricărui controller. Mai apoi, se pot încărca fișiere de tip model cu funcția `$this->loadModel` și se pot afișa fișiere de tip view cu ajutorul `$this->loadView`. Astfel, dezvoltarea de pagini și funcționalități noi în cadrul proiectului este simplă.

Autentificare

În `system`, am implementat clasa `User`, care stochează datele și rolul unui utilizator autentificat, verificând sesiunea HTTP la fiecare încărcare a unei pagini noi sau unui request AJAX.
Grupul de controllere `account` conține controllere pentru înregistrare, login, logout, resetarea parolei și actualizarea datelor contului.
În baza de date, se regăsește tabela `users`, care stochează e-mailul, parola și rolul unui utilizator. La resetarea parolei, se trimite un e-mail cu un link de resetare către mailul utilizatorului. Un utilizator își poate actualiza datele contului.
Utilizatorii cu rol de administrator pot vedea toți utilizatorii și îi pot edita.

Audit

În folderul `system`, am implementat clasa `Audit`, care permite adăugarea intrărilor noi de audit în baza de date, în cadrul tabelei `audit_logs`.

Tichete

Sistemul de tichete este implementat utilizând un controller `tickets/ticket`. Utilizatorii pot crea tichete noi și își pot vedea propriile tichete și statusul lor, iar analiștii și

administratorii pot să modifice statusul tichetelor și să vadă lista întreagă de tichete existente. La actualizarea statusului unui tichet, utilizatorul creator primește un e-mail. Tichetele sunt stocate în tabela `tickets`.

Autorizare

Autorizarea se face cu ajutorul unor variabile globale, `$admin` și `$analyst`, care permit sau restricționează afișarea anumitor linkuri sau date pe baza rolului utilizatorului conectat. Astfel, doar un analist poate vedea toate tichetele, pe când un utilizator normal vede doar tichetele sale, iar un administrator poate accesa lista tuturor utilizatorilor. De asemenea, rolurile sunt ierarhice, așadar un administrator este, automat, și un analist.

Vulnerabilități

Varianta inițială a aplicației prezintă diverse vulnerabilități de autentificare și autorizare, făcând-o o aplicație neutilizabilă pentru un scenariu real, public. În continuare, pentru fiecare vulnerabilitate identificată, vom prezenta cum poate fi performat un atac pe baza ei, impactul acesteia și modul de rezolvare.

Stocarea parolelor

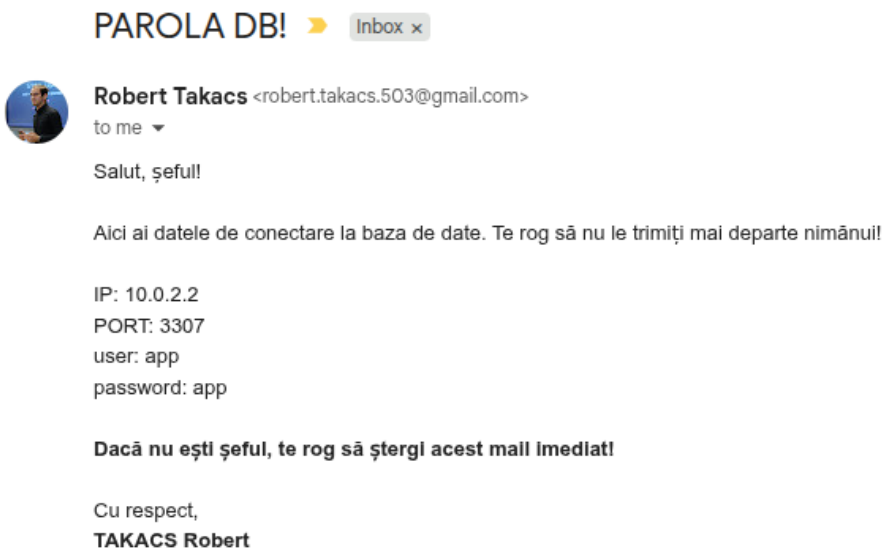
Parolele sunt stocate în format cleartext în baza de date a aplicației.

Riscuri

În cazul în care un atacator obține acces la baza de date, acesta se poate conecta la orice cont în cadrul aplicației, impersonând utilizatorii. De asemenea, mulți utilizatori refolosesc parole în cadrul mai multor aplicații, deci este posibil ca atacatorul să obțină acces la mai multe conturi ale utilizatorului.

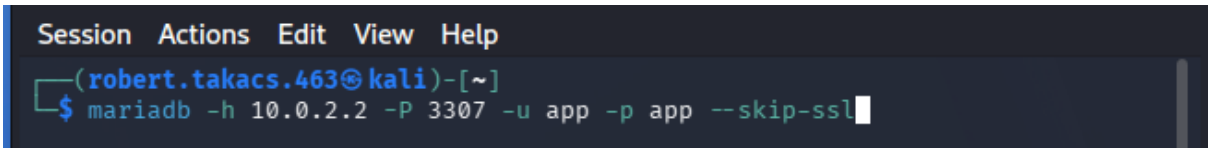
Demonstrație Atac

Ipotetic, prin noroc sau printr-un atac de tip phishing, atacatorul obține datele de conectare la baza de date, care nu implementează autentificare și pe baza IP-ului sau altfel.



Imagine 2. Datele de conectare la baza de date

Atacatorul nostru poate acum să se conecteze la baza de date și are acces deplin la ea.



Imagine 3. Conectarea la baza de date

```
Session  Actions  Edit  View  Help
┌──(robert.takacs.463㉿kali)-[~]
└─$ mariadb -h 10.0.2.2 -P 3307 -u app -p app --skip-ssl
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 56
Server version: 8.0.36 MySQL Community Server - GPL

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement
.

MySQL [app]>
```

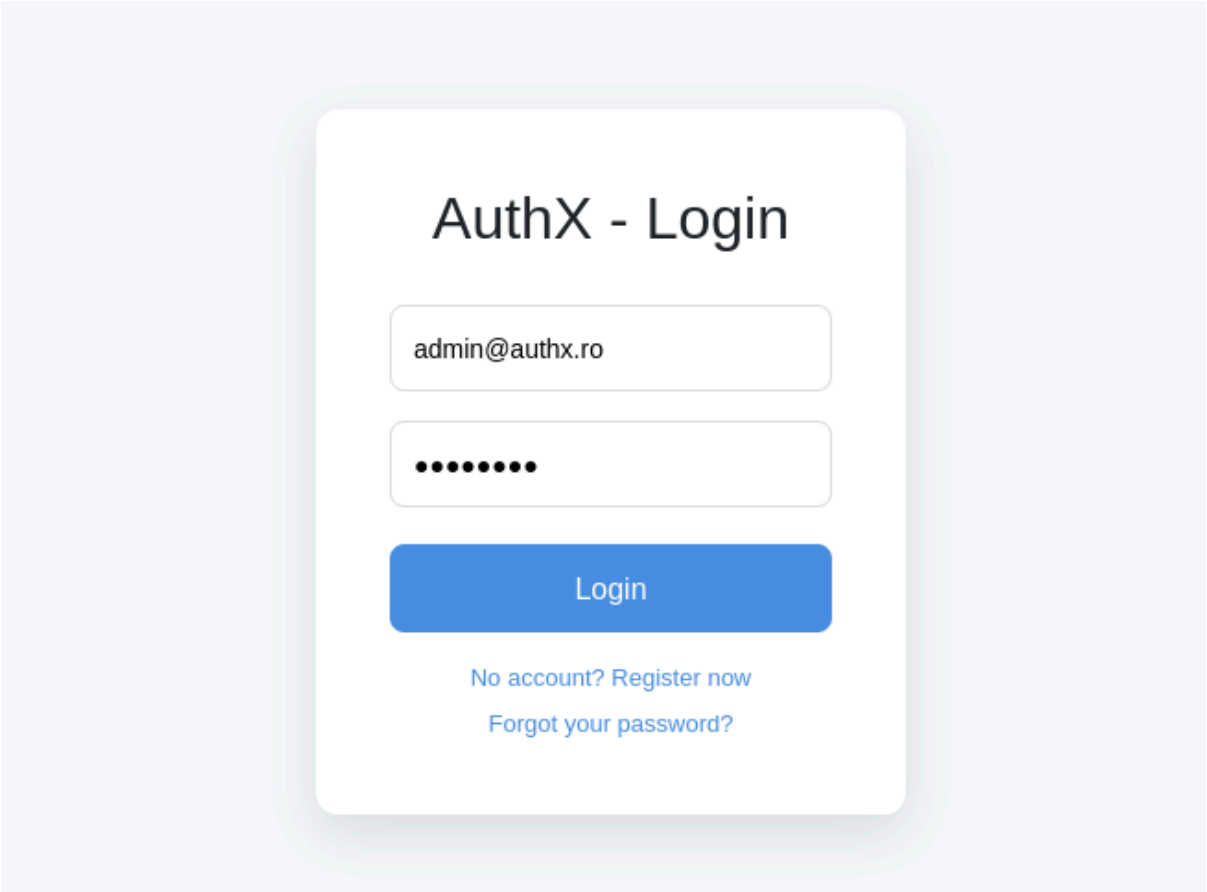
Imagine 4. Conectat la baza de date

Conectat la baza de date, atacatorul poate să identifice tabelele existente, să le analize, și inclusiv să le modifice, dacă contul de SQL la care a obținut acces are aceste drepturi.

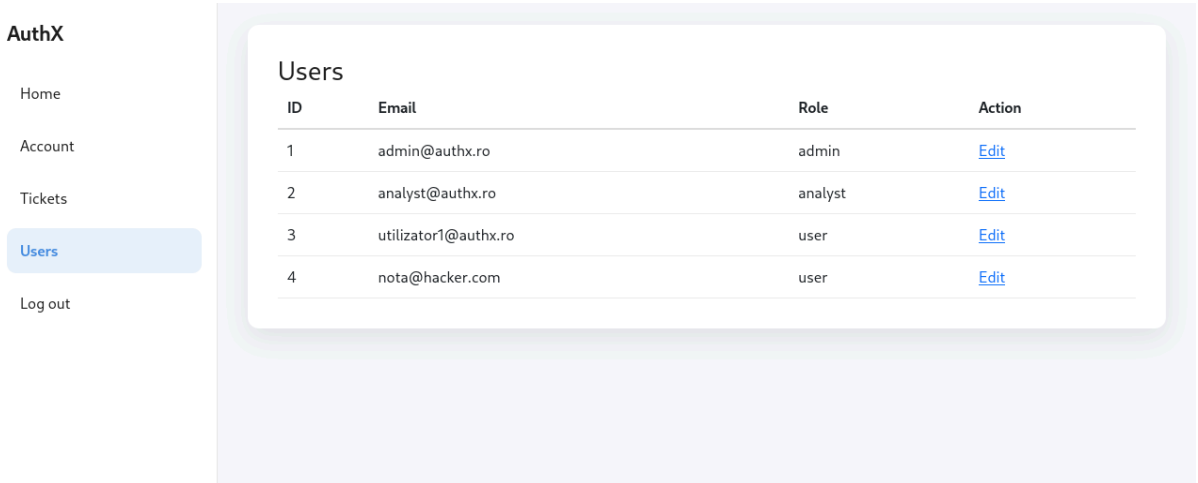
```
Session  Actions  Edit  View  Help
.

MySQL [app]> show tables;
+---------------+
| Tables_in_app |
+---------------+
| audit_logs    |
| tickets       |
| users         |
+---------------+
3 rows in set (0.001 sec)

MySQL [app]> select * from users
    -> ;
+---------+----------------------+-------------+---------+---------------------
--+
| user_id | email                | password    | role    | created_at
  |
+---------+----------------------+-------------+---------+---------------------
--+
|       1 | admin@authx.ro       | password    | admin   | 2026-04-18 06:38:4
7 |
|       2 | analyst@authx.ro     | password    | analyst | 2026-04-18 06:38:4
7 |
|       3 | utilizator1@authx.ro | donthackme! | user    | 2026-04-18 06:42:1
1 |
|       4 | nota@hacker.com      | s3cur3!     | user    | 2026-04-18 06:42:3
8 |
```

Imagine 5. Acces deplin la lista de utilizatori, inclusiv datele lor de conectare.

Problema majoră este dată de faptul că parolele conturilor sunt stocare în cleartext. așadar atacatorul identifică contul de administrator și se conectează fără probleme la acesta.



Imagine 6. Atacatorul se poate conecta la aplicație cu contul de administrator



Imagine 7. Atacatorul obține acces deplin la aplicație

Conectat la contul de administrator, atacatorul are acces deplin la aplicație.

Impact

Expunerea parolelor utilizatorilor compromite atât confidențialitatea, cât și integritatea datelor din cadrul aplicației, deci riscul este unul major și nu poate fi ignorat.

Rezolvare

Parolele se stochează doar utilizând algoritmi începți de hashing, în special aceia care integrează și un sistem de salting. Pentru securitate, stocăm parolele doar cu algoritmul bcrypt, utilizând funcția password_hash din PHP, iar pentru orice verificare a parolei, folosim funcția password_verify. Utilizând algoritmi începți, prevenim atacuri de tip brute-forcing, iar algoritmii cu salting încorporat rezolvă și problema atacurilor care folosesc rainbow tables (tabele curcubeu).

```php
public function register($email, $password)
{
    $password_hash = password_hash($password, PASSWORD_BCRYPT);

    $sql = "INSERT INTO users (email, password) VALUES (?, ?)";
    $result = $this->db->query($sql, 'ss', [$email, $password_hash]);

    $this->session->data['user_id'] = $result->insert_id;
    $this->user_id = $this->session->data['user_id'];
    $this->email = $email;
    $this->role = 'user';

    return true;
}
```

Imagine 8. Modificarea funcției register

```php
public function login($email, $password)
{
    $sql = "SELECT * FROM users WHERE email = ?";
    $result = $this->db->query($sql, 's', [$email]);

    if ($result->num_rows > 0) {
        $user = $result->row;

        if (password_verify($password, $user['password'])) {
            $this->session->data['user_id'] = $user['user_id'];
            $this->user_id = $user['user_id'];
            $this->email = $user['email'];
            $this->role = $user['role'];
            return true;
        }
    }

    return false;
}
```

Imagine 9. Modificarea funcției login

Cerințe de securitate ale parolelor

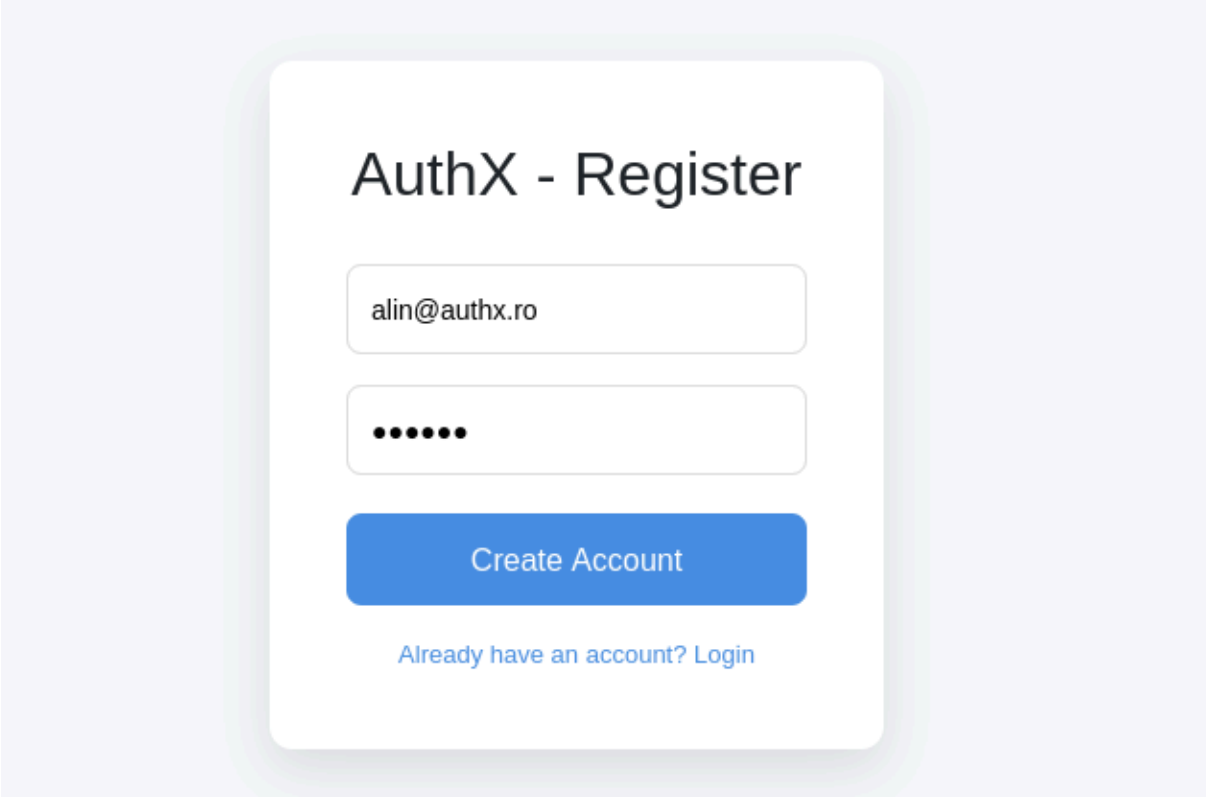
La crearea sau modificarea unei parole, nu există restricții legate de lungimea sau complexitatea unei parole.

Riscuri

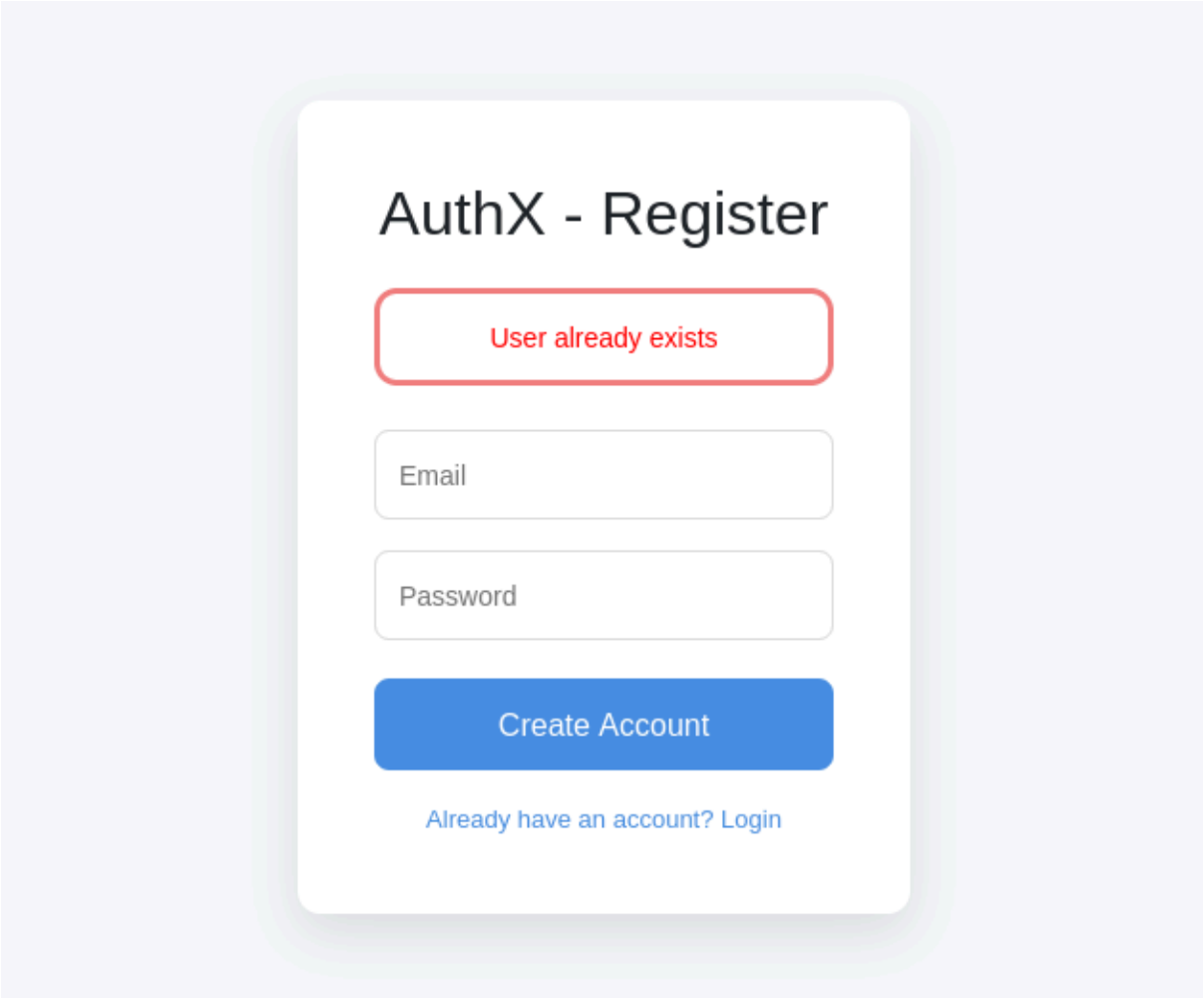
Parolele prea simple, cum ar fi 123456 sau 000000, pot fi ghicite ușor de către atacatori, aceștia putându-se conecta și impersona alți utilizatori dacă le ghicesc parola. Unelte de brute force conțin baze date date de parole comune și simple, și le pot ghici în câteva milisecunde.

Demonstrație Atac

Atacatorul poate identifica prin metodele de user enumeration conturi care există, de exemplu, cunoaște e-mailul unui angajat, Alin, care este administrator al companiei, și verifică dacă acesta are cont utilizând funcționalitatea de înregistrare.



Imagine 10. Înregistrare cu e-mail-ul lui Alin



Imagine 11. Alin are deja cont, și cu puțin noroc îi putem identifica contul

La încercarea de înregistrare cu contul cunoscut, atacatorul observă că utilizatorul există, și deci poate începe să-i caute parola utilizând algoritmi de bruteforcing.

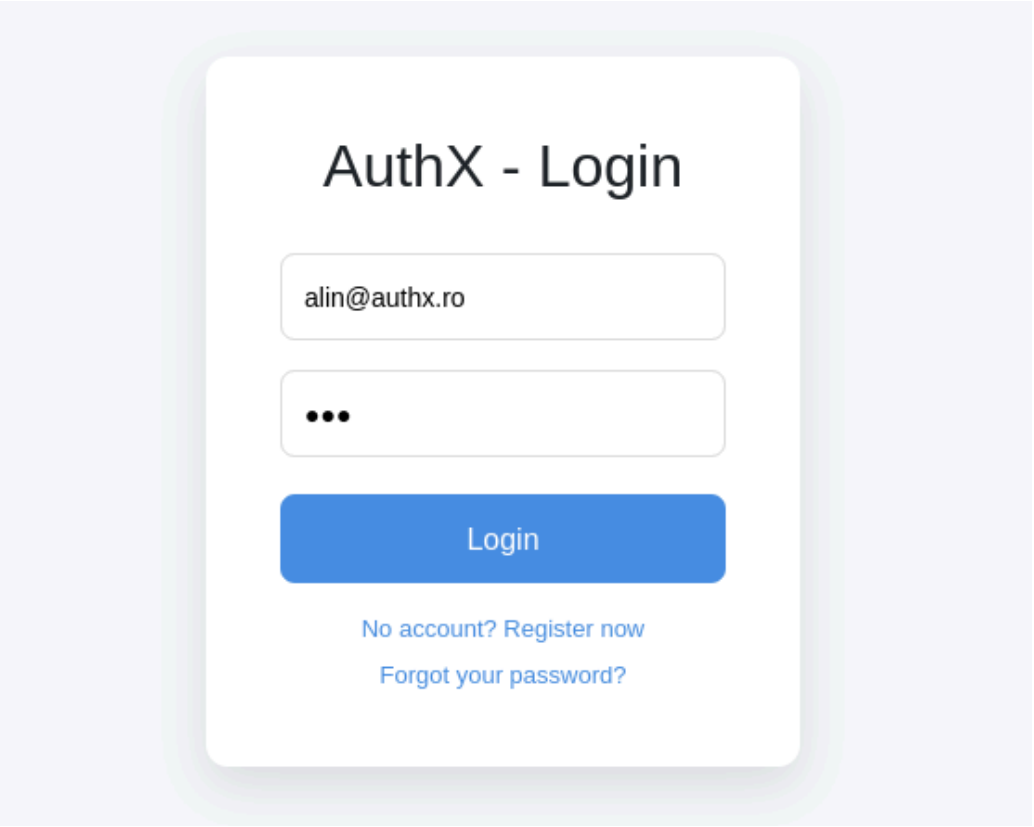
```bash
#!/usr/bin/env bash

URL="http://10.0.2.2:8080/account/login/submit"
EMAIL="alin@authx.ro"
ATTEMPTS=1000

for i in $(seq 1 "$ATTEMPTS"); do
  PASSWORD="$i"
  response_headers=$(
    curl -s -D - -o /dev/null \
      -X POST "$URL" \
      -H "Content-Type: application/x-www-form-urlencoded" \
      --data-urlencode "email=$EMAIL" \
      --data-urlencode "password=$PASSWORD"
  )
        location=$(printf '%s\n' "$response_headers" | grep -i '^Location:' | tr -d '\r' | cut -d' ' -f2-)

        if [ "$location" = "/common/home" ]; then
                echo "success: password $PASSWORD"
        fi

        if (( i % 10 == 0 )); then
                echo "Reached $i attempts"
        fi
done
```

Imagine 12. Script naiv de bruteforcing

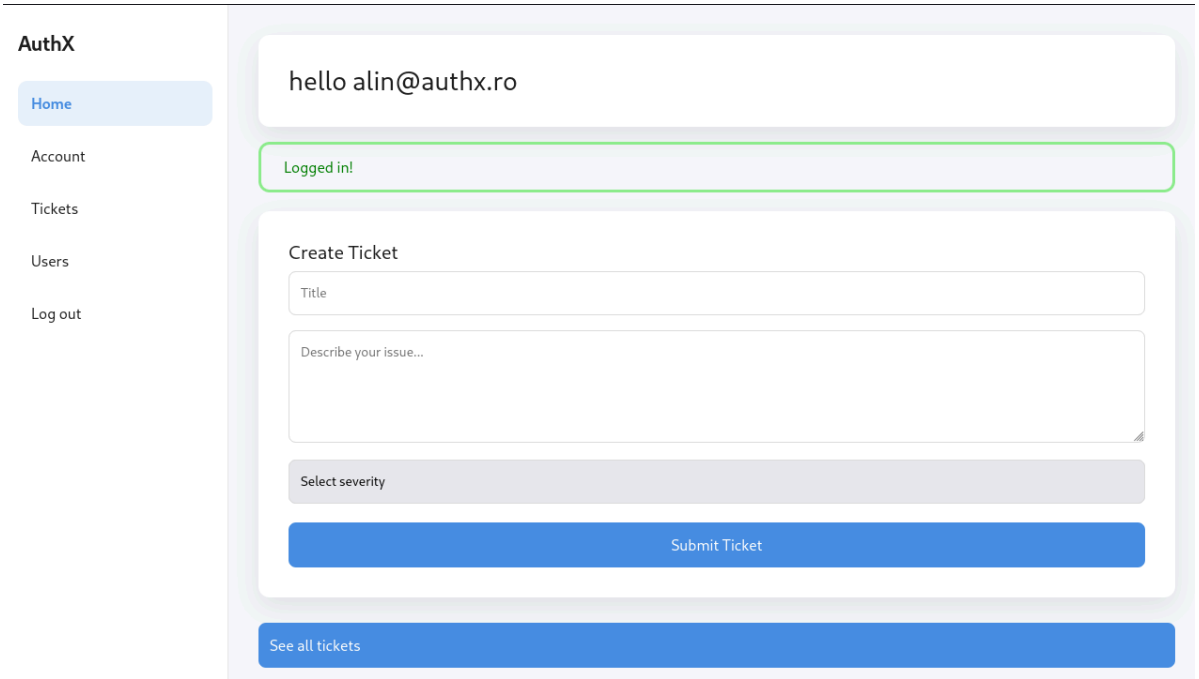
```
┌──(robert.takacs.463㉿kali)-[~/Desktop/proiect]
└─$ ./bf.sh
Reached 10 attempts
Reached 20 attempts
Reached 30 attempts
Reached 40 attempts
Reached 50 attempts
Reached 60 attempts
Reached 70 attempts
Reached 80 attempts
Reached 90 attempts
Reached 100 attempts
Reached 110 attempts
Reached 120 attempts
success: password 123
```

Imagine 13. Rezultatul scriptului de brute-forcing

După puține încercări, descoperim parola utilizatorului. Acum. atacatorul se poate conecta la contul administratorului și obține acces deplin la aplicație.



Imagine 14. Atacatorul se conectează la contul de administrator



Imagine 15. Și obține acces deplin la aplicație

Impact

Permiterea utilizării parolelor nesigure duce la riscuri de acces neautorizat la conturile utilizatorilor, compromițând confidențialitatea și integritatea datelor din cadrul aplicației.

Rezolvare

Parolele utilizatorilor conțin cerința de a avea lungimea minimă de 10 caractere. Nu se impun alte restricții sau cerințe legate de lungimea parolelor, sau sisteme de resetare automată a parolei la anumite intervale de timp, deoarece acestea nu îmbunătățesc securitatea unei parole. În general, parolele cele mai sigure sunt compuse din mai multe cuvinte aleatorii.

```
class AccountRegisterController extends BaseController
    public function submit()
            return;
        }

        if (strlen($this->request->post['password']) < 10) {
            $this->notification->set('error', 'Password must be at least
            10 characters long');
            $this->response->redirect('account/register');
            return;
        }
```

Imagine 16. Adăguarea validării la înregistrare

```
class AccountAccountController extends BaseController
    public function edit()
                return;
            }

            if (strlen($new_password) < 10) {
                $this->notification->set('error', 'New password must be
                at least 10 characters long');
                $this->response->redirect('account/account');
                return;
            }
```

Imagine 17. Adăguarea validării la editare cont

```
<script>
    document.querySelector('.auth-form').addEventListener('submit',
    function (e) {
        const password = document.querySelector('input[name="password"]').
        value;

        if (password.length < 10) {
            e.preventDefault();

            let notif = document.querySelector('.
            auth-card__notification');
            if (!notif) {
                notif = document.createElement('div');
                notif.className = 'auth-card__notification';
                document.querySelector('.auth-card').prepend(notif);
            }

            notif.innerText = 'Password must be at least 10 characters
            long';
        }
    });
</script>
```

Imagine 18. Adăguarea unui mesaj de validare în javascript la înregistrare

Resetarea Parolelor

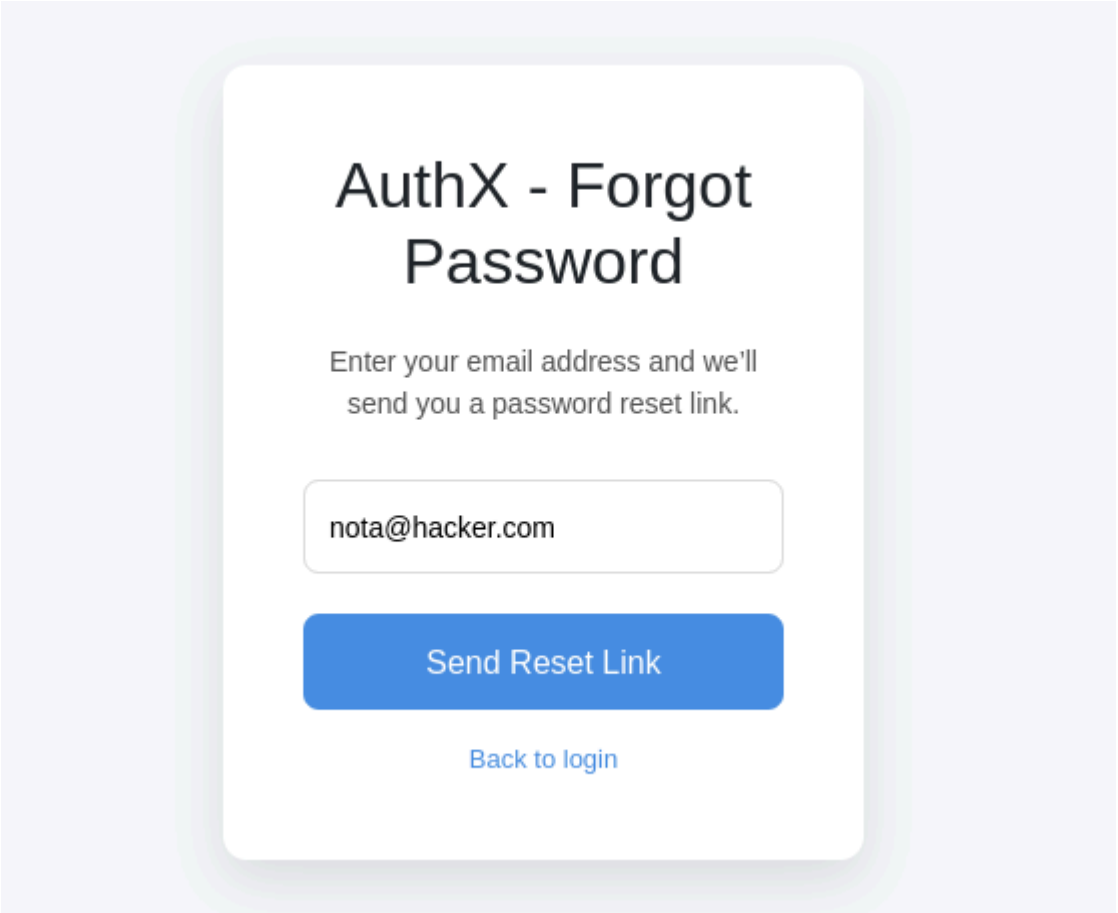
Resetarea contului se poate face prin accesarea unui link static per utilizator, incluzând doar ID-ul utilizatorului.

Riscuri

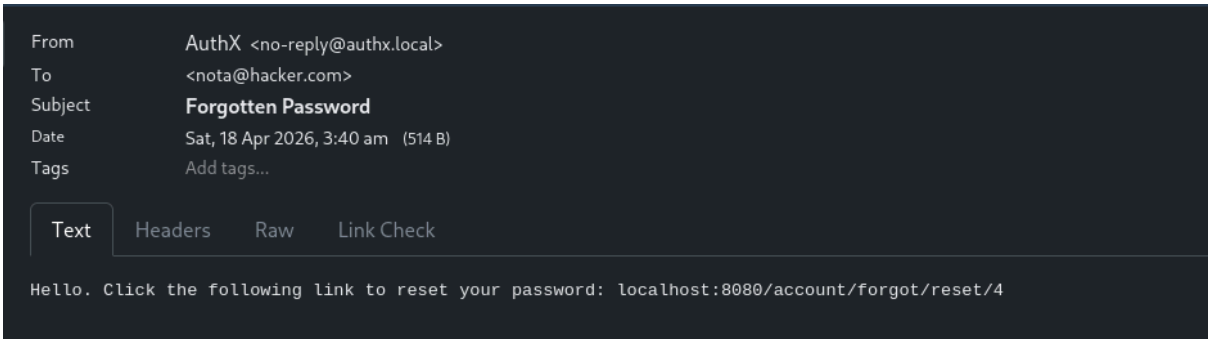
Odată ce un utilizator cu intenții malițioase înțelege că poate modifica ID-ul din linkul de resetare a contului, acesta poate actualiza contul oricărui utilizator. Dacă reușește, de asemenea, să identifice e-mailul (de exemplu, dacă baza de date este compromisă), acesta se va putea conecta la conturile altor utilizatori. Utilizatorul poate, de asemenea, să vadă, pe baza răspunsului site-ului la cererea de resetare, dacă un cont există sau nu, răspunsul fiind fie „User does not exist", fie pagina de „e-mail trimis".

Demonstrație Atac

Atacatorul începe prin a experimenta cu sistemul de resetare a parolei, pentru a analiza cum funcționează.

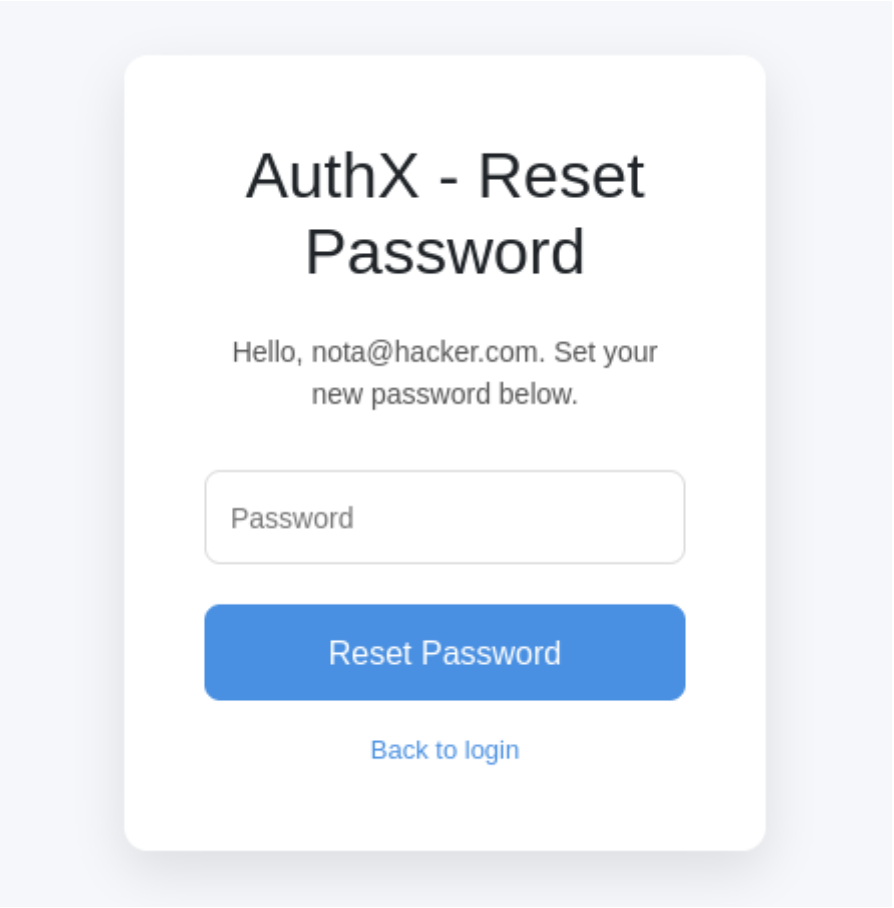


Imagine 19. Atacatorul experimentează cu sistemul de resetare parolă



Imagine 20. E-mailul de resetare parolă

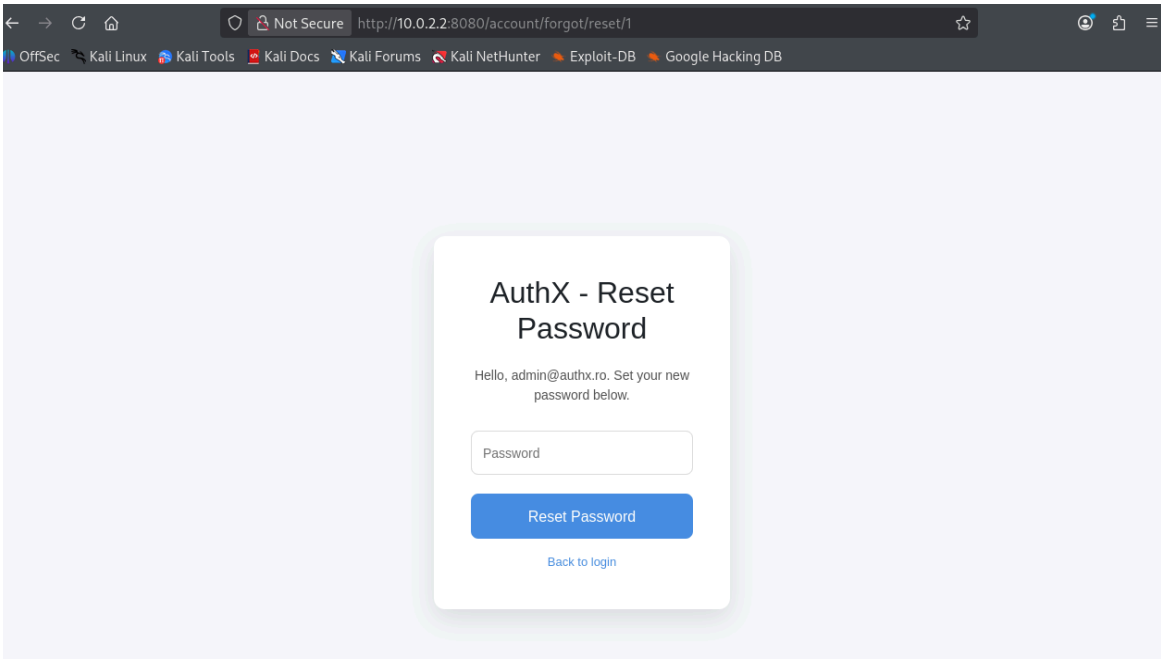
La primirea e-mailului de resetare, atacatorul observă că link-ul de resetare include un număr care pare a fi ID-ul utilizatorului pentru care se resetează parola.



Imagine 21. Pagina de resetare parolă

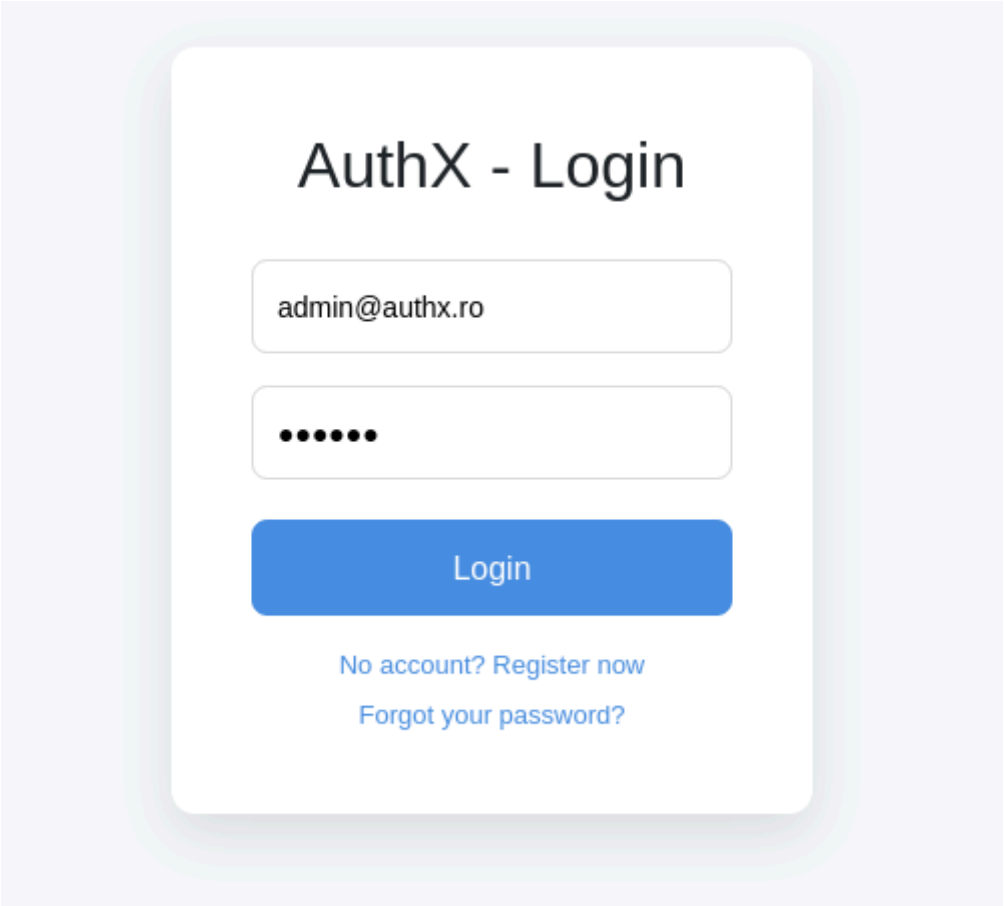
Imagine 22. URL resetare parolă

Modificând manual URL-ul paginii, atacatorul schimbă ID-ul „4” în „1” și este trimis pe pagina de resetare a contului de administrator. Acum, tot ce trebuie să facă este să schimbe parola.

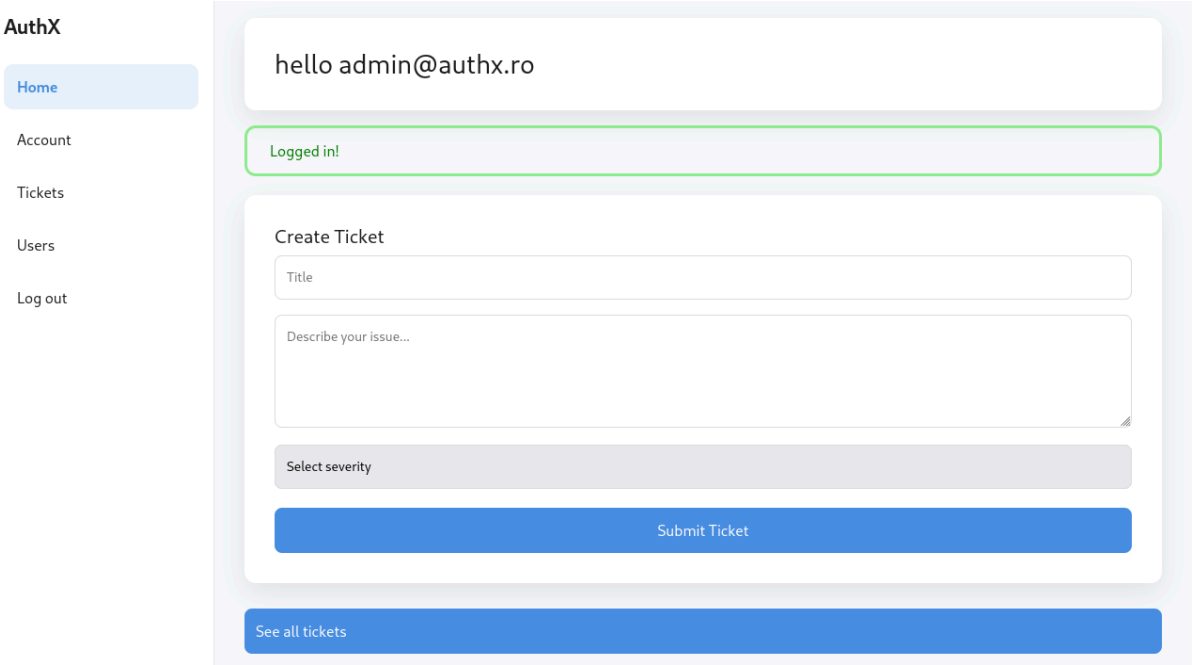


Imagine 23. Modificarea URL-ului permite actualizarea contului de administrator

După ce parola este modificată, atacatorul se poate conecta simplu la contul de administrator.



Imagine 24. Conectarea la admin cu parola nouă



Imagine 25. Atacatorul obține acces deplin la aplicație

Impact

Asemenea cazurilor anterioare, confidențialitatea și integritatea datelor aplicației sunt compromise dacă atacatorul obține acces la conturile altor utilizatori. Pe de altă parte, prin modificarea parolei unui cont, atacatorul întrerupe și accesul utilizatorului legitim, iar mai apoi poate modifica e-mailul contului, blocând complet accesul la cont, deci afectând și disponibilitatea platformei.

Rezolvare

Această problemă de securitate se rezolvă prin implementarea unui sistem de resetare care utilizează token-uri one-time care expiră după scurt timp. Astfel, fie că e-mailul cerut există sau nu, utilizatorul este informat că e-mailul cu linkul de resetare s-a trimis către e-mailul cerut. Linkul de resetare funcționează aplicând o funcție de hash unui token format din biți random, securi criptografic (cu funcția random_bytes și hash-ul SHA256). Se inserează în baza de date, în tabela user_reset_tokens, token-ul de resetare, împreună cu o dată de expirare (noi am ales o oră), și se anulează toate token-urile generate anterior pentru acel cont. Doar accesând linkul cu acest token se poate reseta contul utilizatorului, după care tokenul devine inactiv.

```php
class AccountForgotController extends BaseController
    public function submit()
            return;
        }

        $token = bin2hex(random_bytes(32));
        $token_hash = hash('sha256', $token);
        $expires_at = date('Y-m-d H:i:s', time() + 3600);

        $this->db->query(
            "DELETE FROM user_reset_tokens WHERE user_id = ?",
            'i',
            [$existing_user['user_id']]
        );

        $this->db->query(
            "INSERT INTO user_reset_tokens (user_id, token_hash,
            expires_at, created_at) VALUES (?, ?, ?, NOW())",
            'iss',
            [$existing_user['user_id'], $token_hash, $expires_at]
        );

        $reset_link = $this->url->link('account/forgot/reset/' . $token,
        true);

        $this->audit->add('USER_FORGOTTEN', $this->request->post
        ['email'], $existing_user['user_id']);
        $this->mail->send(
            $this->request->post['email'],
            'Forgotten Password',
            'Hello. Click the following link to reset your password: ' .
            $reset_link
        );

        $this->response->redirect('account/forgot/sent');
    }
```

Imagine 26. Crearea unui token de resetare la „Mi-am uitat parola”

```php
class AccountForgotController extends BaseController
    public function reset_submit($setting)
        }

        $token = $setting[0];
        $token_hash = hash('sha256', $token);

        $reset_token = $this->db->query(
            "SELECT * FROM user_reset_tokens WHERE token_hash = ? AND
            used_at IS NULL AND expires_at >= NOW() LIMIT 1",
            's',
            [$token_hash]
        )->row;

        if (!$reset_token) {
            $this->notification->set('error', 'Invalid or expired reset
            link');
            $this->response->redirect('account/forgot');
            return;
        }

        $this->loadModel('account/user');

        $user = $this->model_account_user->getUserById($reset_token
        ['user_id']);

        if (!$user) {
            $this->response->redirect('account/login');
            return;
        }


        $this->model_account_user->updatePassword($user['user_id'],
        $this->request->post['password']);

        $this->db->query(
            "UPDATE user_reset_tokens SET used_at = NOW() WHERE
            reset_token_id = ?",
            'i',
            [$reset_token['reset_token_id']]
        );

        $this->audit->add('USER_RESET_PASSWORD', $user['email'], $user
        ['user_id']);
        $this->notification->set('success', 'Password updated
        successfully');
```

Imagine 27. Validarea token-ului de resetare și modificarea parolei

```
<?= $head ?>

<div class="auth-layout">
    <div class="auth-card">
        <h2 class="auth-card__title">AuthX - Forgot Password</h2>

        <div class="auth-card__text">
            If a user with the specified e-mail exists, we'll send an
            e-mail with the password reset link!
        </div>

        <div class="auth-card__links">
            <a class="auth-card__link" href="<?= $back ?>">Back to login</
            a>
        </div>
    </div>
</div>

<?= $footer ?>
```

Imagine 28. Mesajul de confirmare nu divulgă dacă contul există sau nu

Operații Neautorizate

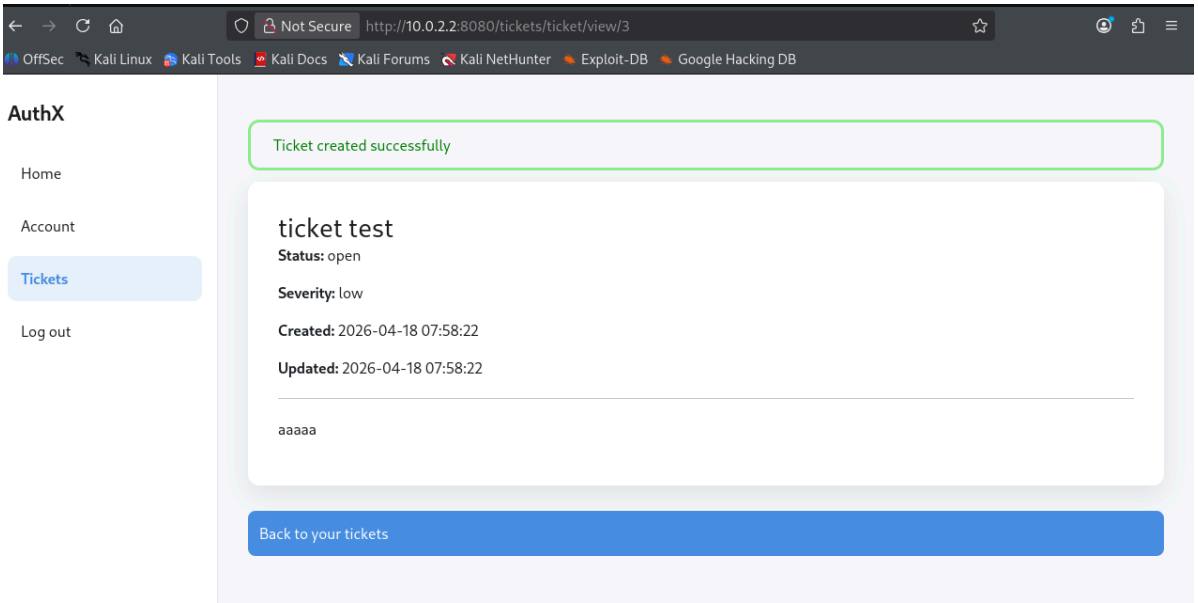
Deși link-urile către pagini sau operațiuni restricționate, cum ar fi listă utilizatori, editare cont, editare tichete, erau afișate doar celor cu drepturi, operațiile și paginile în sine permiteau accesul fără verificare a autorizării. Astfel, orice utilizator poate să acceseze manual link-uri de editare utilizator, vizualizare tichet, sau să trimită un request POST „editare” fără a avea de fapt acces.

Riscuri

Un utilizator cu intenții malițioase poate încerca, pe baza link-urilor sale de vizualizare tichet propriu, să modifice ID-ul tichetului din URL, și va observa că poate accesa tichetele altor utilizatori. Dacă reușește să identifice endpoint-urile pentru editare tichet sau utilizator, va putea mai apoi să modifice aceste date ale oricărui tichet sau utilizator.

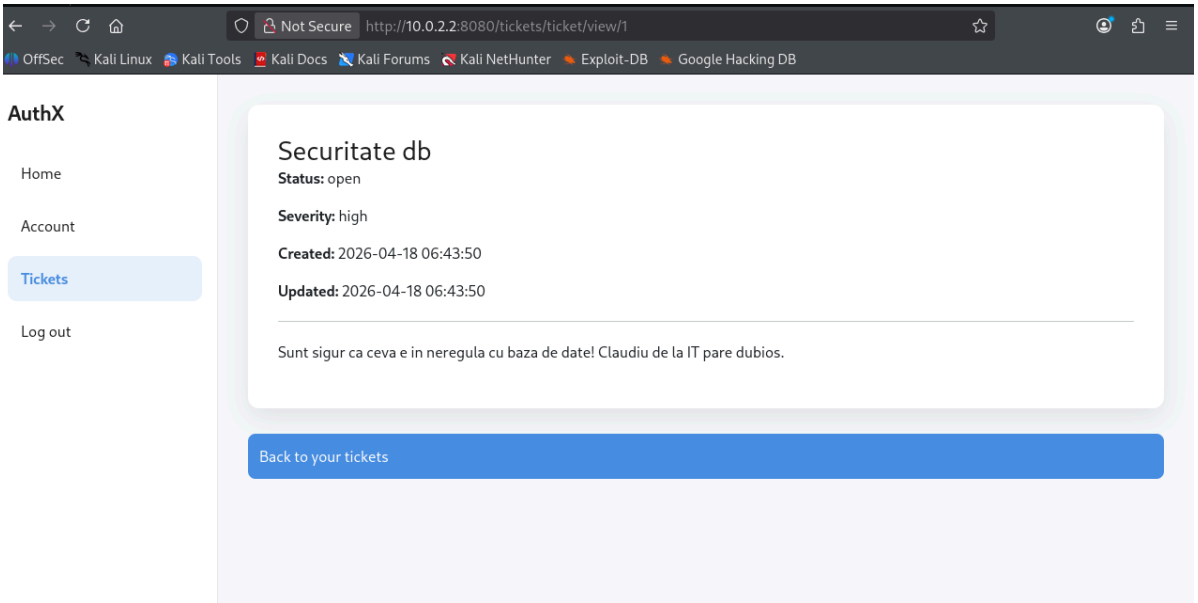
Demonstrație Atac

Un atacator poate experimenta cu sistemul de creare tichete.



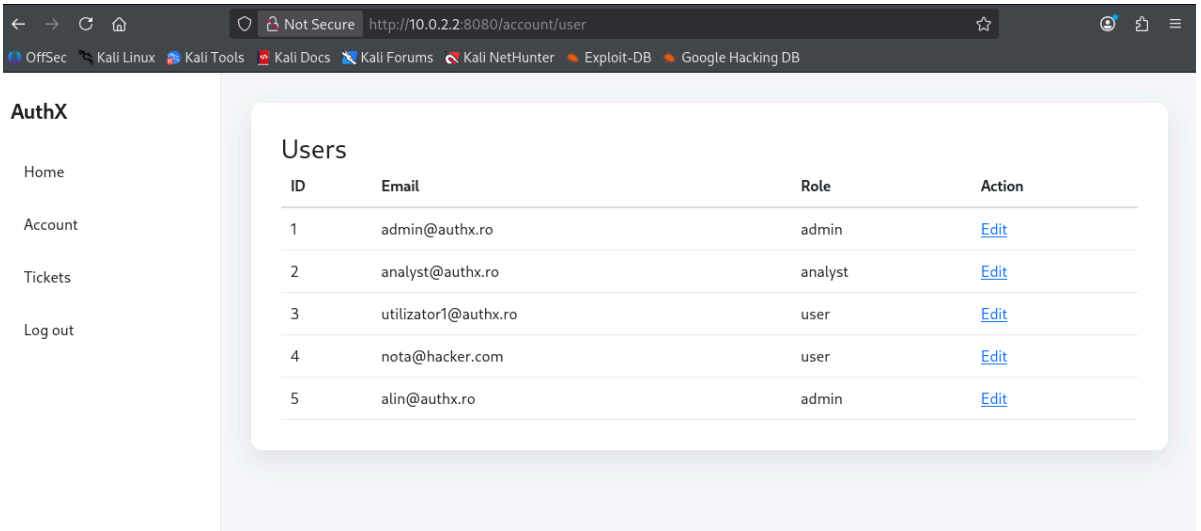
Imagine 29. Atacatorul experimentează cu sistemul de creare tichete

Acesta observă că primul său tichet are în URL numărul „3”, care poate reprezenta un ID. Modificând manual ID-ul, reușește să acceseze un tichet creat de alt utilizator.

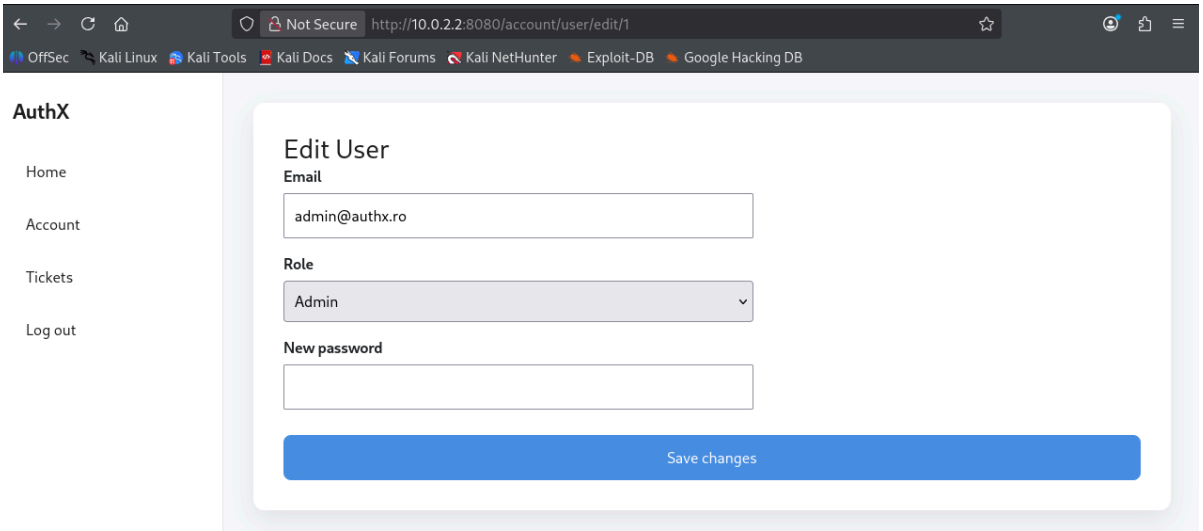


Imagine 30. Atacatorul accesează informațiile unui alt utilizator

Astfel, atacatorul poate accesa toate datele scrise de alți utilizatori și să le urmărească statusul. De asemenea, dacă atacatorul identifică url-ul reprezentând lista de utilizatori ai aplicației, o poate accesa și poate chiar să editeze conturile altor utilizatori.



Imagine 31. Atacatorul accesează lista de utilizatori



Imagine 32. Atacatorul poate edita contul oricărui utilizator

Impact

Această problemă implică, din nou, probleme de confidențialitate a datelor (un utilizator poate vedea datele tuturor tichetelor chiar dacă nu ar trebui), dar și de integritate a datelor, acesta putând să modifice informațiile unui alt utilizator sau a unui tichet.

Rezolvare

Pe fiecare operație care necesită autorizare, și la fiecare controller asociat acestor operații, se efectuează verificări de autorizare (un utilizator poate vedea un tichet doar dacă e al său sau e analist sau admin, un utilizator poate edita un tichet doar dacă e analist sau admin, listă utilizatori doar dacă e admin, pagină editare utilizator și acțiune editare utilizator doar dacă e admin).

```
class AccountUserController extends BaseController
    public function index()

        if (!$this->user->isAdmin()) {
            $this->notification->set('error', 'Unauthorized');
            $this->response->redirect('common/home');
            return;
        }
```

Imagine 33. Autorizare pe lista de utilizatori

```
class AccountUserController extends BaseController
    public function edit($setting)

        if (!$this->user->isAdmin()) {
            $this->notification->set('error', 'Unauthorized');
            $this->response->redirect('common/home');
            return;
        }
```

Imagine 34. Autorizare pe pagina de editare utilizator

```
class AccountUserController extends BaseController
    public function update($setting)

        if (!$this->user->isAdmin()) {
            $this->notification->set('error', 'Unauthorized');
            $this->response->redirect('common/home');
            return;
        }
```

Imagine 35. Autorizare la editarea unui utilizator

```
class TicketsTicketController extends BaseController
    public function view($setting)
        }

        if ($ticket['owner_id'] != $this->user->loggedIn() && !
        ($this->user->isAdmin() || $this->user->isAnalyst())) {
            $this->notification->set('error', 'Unauthorized');
            $this->response->redirect('common/home');
            return;
        }
```

Imagine 36. Autorizare pe pagina de tichet

```
class TicketsTicketController extends BaseController
    public function update($setting)
        }

        if (!$this->user->isAdmin() || !$this->user->isAnalyst()) {
            $this->notification->set('error', 'Unauthorized');
            $this->response->redirect('common/home');
            return;
        }
```

Imagine 37. Autorizare la editarea unui tichet

Brute-forcing

Nu există limite pentru operațiile de tip login. Astfel, un request de tip login poate fi refăcut într-o perioadă scurtă de timp de un număr nelimitat de ori.

Riscuri

Dacă un utilizator rău-intenționat cunoaște e-mailul unui alt utilizator și dorește să obțină acces la contul acestuia, poate să itereze un număr extrem de mare de request-uri către endpoint-ul de login al aplicației, enumerând parolele posibile până când reușește să se conecteze la contul dorit.

Demonstrație Atac

Asemeni situației prezentate la problema de securitate ale parolelor utilizatorilor, un atacator poate să facă un număr nelimitat de request-uri către aplicație într-un timp foarte scurt, putând crea atacuri de tip Denial-of-Service. Pentru imaginea de mai jos, a fost folosit script-ul din imaginea 9.

```
┌──(robert.takacs.463㉿kali)-[~/Desktop/proiect]
└─$ ./bf.sh
Reached 10 attempts
Reached 20 attempts
Reached 30 attempts
Reached 40 attempts
Reached 50 attempts
Reached 60 attempts
Reached 70 attempts
Reached 80 attempts
Reached 90 attempts
Reached 100 attempts
Reached 110 attempts
Reached 120 attempts
success: password 123
```

Imagine 38). Brute-forcing pe endpoint-ul de login

Impact

Accesul neautorizat la conturi reprezintă o breșă de confidențialitate și de integritate a datelor aplicației. Pe de altă parte, un număr nelimitat de request-uri către o aplicație poate acționa și ca un atac Denial-of-Service, întrerupând disponibilitatea aplicației.

Rezolvare

Utilizând tabela audit_logs, unde se marchează de fiecare dată când un utilizator cu un anume IP eșuează la login, permitem doar 5 request-uri de login în 10 minute de la un IP specific. Astfel, nu mai este posibil un atac rapid de tip brute-force, deoarece parolele nu pot fi enumerate la o viteză îndeajuns de mare.

```php
    private function isLoginBlockedByIp($ip)
    {
        $result = $this->db->query(
            "SELECT COUNT(*) AS total
         FROM audit_logs
         WHERE action = ?
           AND ip_address = ?
           AND timestamp >= DATE_SUB(NOW(), INTERVAL 10 MINUTE)",
            'ss',
            ['USER_LOGIN_FAILED', $ip]
        );

        return (int) $result->row['total'] >= 5;
    }
}
```

Imagine 39. Funcția de verificare a brute-forcing-ului la login

```php
class AccountLoginController extends BaseController
    public function submit()
    {
        if ($this->request->server['REQUEST_METHOD'] != 'POST') {
            $this->response->redirect('account/login');
            return;
        }

        $ip = $this->request->server['REMOTE_ADDR'];

        if ($this->isLoginBlockedByIp($ip)) {
            $this->notification->set('error', 'Too many failed login
            attempts. Try again later.');
            $this->response->redirect('account/login');
            return;
        }
```

Imagine 40. Verificarea brute-forcing-ului la login

Enumerare de conturi

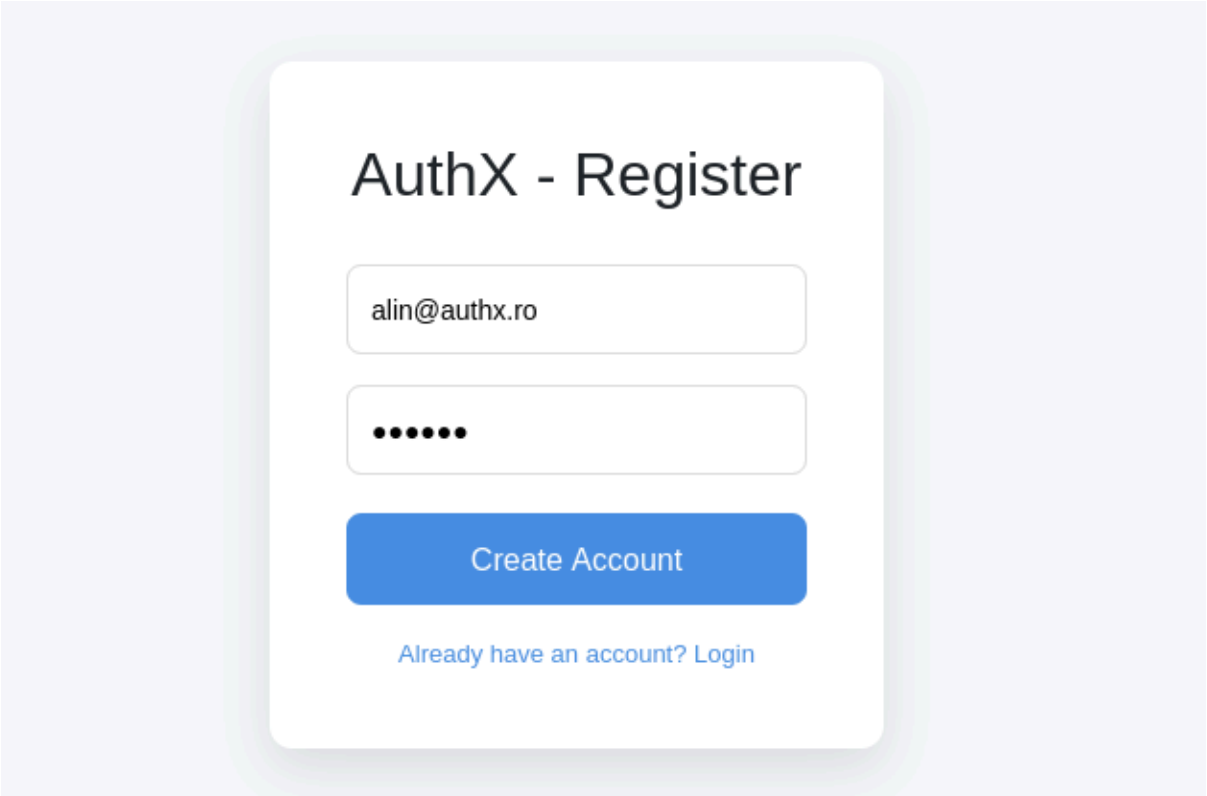
La înregistrare, utilizatorul este informat dacă contul cu e-mailul folosit există deja. Contul se creează dacă nu există deja sau eșuează cu informare dacă contul există deja.

Riscuri

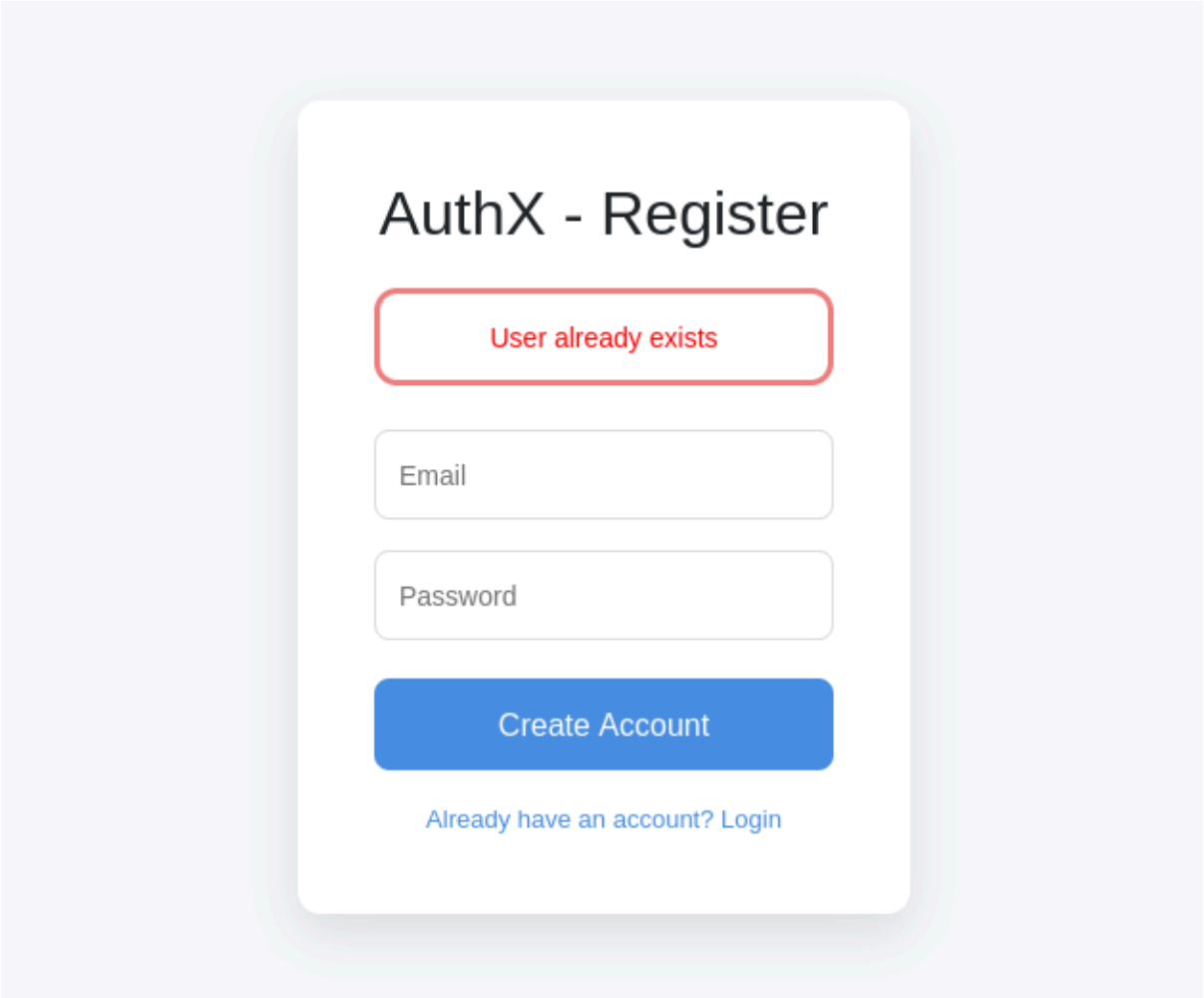
Acest mecanism implică riscul enumerării utilizatorilor aplicației. Un utilizator rău-intenționat, care cunoaște un set de e-mailuri, poate verifica care utilizatori au cont pe aplicație și care nu.

Demonstrație Atac

Atacatorul poate identifica e-mailuri care au conturi asociate, și e-mailuri pentru care nu există conturi. De exemplu, dacă apelează endpointul de înregistrare cu un email care există, aplicația îl informează că utilizatorul deja există.

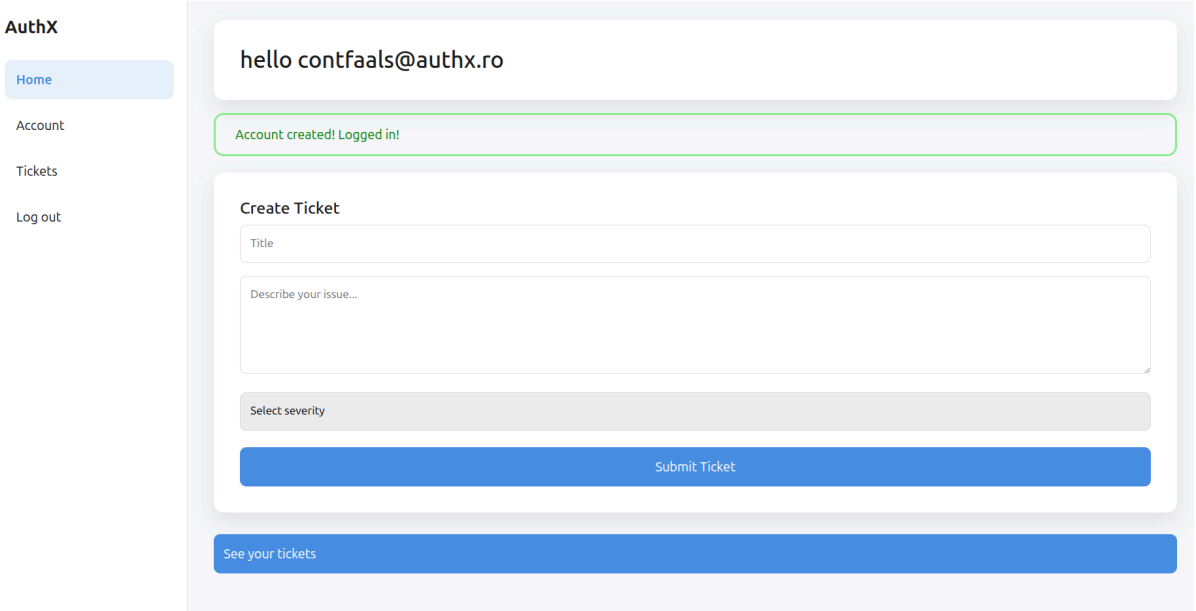


Imagine 41. Înregistrare cu e-mail-ul lui Alin



Imagine 42. Alin are cont

Pe de altă parte, dacă încearcă un e-mail fără cont asociat, mesajul este diferit, acela de „contul a fost creat”.



Imagine 43. Contul a fost creat

Impact

Enumerarea utilizatorilor reprezintă o afectare a confidențialității aplicației, în situația în care este imperativ ca existența conturilor să fie una anonimă. Pe de altă parte, utilizatorul poate să creeze un număr mare de conturi fără limită.

Rezolvare

Implementăm un sistem de verificare a conturilor cu ajutorul e-mailului la înregistrare. Astfel, mesajul după înregistrare este întotdeauna același - „dacă e-mailul există, ți-am trimis un mail cu următorii pași". Dacă contul nu există sau există dar nu este verificat, mailul conține un link de verificare a contului. Altfel, mailul conține un link de resetare a parolei pentru a ajuta utilizatorul să-și recupereze contul. Utilizatorul de frontend nu mai poate cunoaște dacă contul există sau nu fără să acceseze mail-ul.

```php
class AccountRegisterController extends BaseController
    public function submit()
        $this->loadModel('account/user');

        $existing_user = $this->model_account_user->getUserByEmail
        ($email);

        if (!$existing_user) {
            $user_id = $this->user->register($email, $password);

            $token = bin2hex(random_bytes(32));
            $token_hash = hash('sha256', $token);
            $expires_at = date('Y-m-d H:i:s', time() + 3600 * 24);

            $this->model_account_user->setVerificationToken($user_id,
            $token_hash, $expires_at);

            $verify_link = $this->url->link('account/register/verify/' .
            $token, true);

            $this->mail->send(
                $email,
                'Verify your AuthX account',
                'Hello. Click the following link to verify your account:
                ' . $verify_link
            );

            $this->audit->add('USER_REGISTER', $email, $user_id);
```

Imagine 44. La înregistrare, dacă contul nu există se trimite un email cu un link de verificare

```
        } else {
            if (!$existing_user['is_verified']) {
                $token = bin2hex(random_bytes(32));
                $token_hash = hash('sha256', $token);
                $expires_at = date('Y-m-d H:i:s', time() + 3600 * 24);

                $this->model_account_user->setVerificationToken
                ($existing_user['user_id'], $token_hash, $expires_at);

                $verify_link = $this->url->link('account/register/verify/
                ' . $token, true);

                $this->mail->send(
                    $email,
                    'Verify your AuthX account',
                    'Hello. Click the following link to verify your
                    account: ' . $verify_link
                );
            } else {
                $this->mail->send(
                    $email,
                    'Your AuthX account',
                    'Hello. You already have a verified account. If you
                    want to reset your password, click the following
                    link: ' . $this->url->link('account/forgot', true)
                );
            }

            $this->audit->add('USER_REGISTER', $email, 0);
        }

        $this->notification->set('success', 'Before logging in, check
        your email to verify your account.');
        $this->response->redirect('account/login');
    }
```

Imagine 45. Dacă contul există deja, se trimite fie un link de verificare sau un link de „mi-am uitat parola”

```
class AccountRegisterController extends BaseController
    public function verify($setting)
    {
        if (empty($setting[0])) {
            $this->notification->set('error', 'Invalid verification
            link');
            $this->response->redirect('account/login');
            return;
        }

        $token = $setting[0];
        $token_hash = hash('sha256', $token);

        $this->loadModel('account/user');

        $user = $this->model_account_user->getUserByVerificationToken
        ($token_hash);

        if (!$user) {
            $this->notification->set('error', 'Invalid or expired
            verification link');
            $this->response->redirect('account/login');
            return;
        }

        $this->model_account_user->verifyUser($user['user_id']);

        $this->audit->add('USER_VERIFY', $user['email'], $user
        ['user_id']);
        $this->notification->set('success', 'Account verified. You can
        now log in.');

        $this->response->redirect('account/login');
    }
}
```

Imagine 46. Prin endpoint-ul de verificare se activează contul

```php
class User

    public function login($email, $password)
    {
        $sql = "SELECT * FROM users WHERE email = ?";
        $result = $this->db->query($sql, 's', [$email]);

        if ($result->num_rows > 0) {
            $user = $result->row;

            if (password_verify($password, $user['password'])) {

                if (!$user['is_verified']) {
                    return false;
                }
                $this->session->data['user_id'] = $user['user_id'];
                $this->user_id = $user['user_id'];
                $this->email = $user['email'];
                $this->role = $user['role'];
                return true;
            }
        }

        return false;
    }
```

Imagine 47. Login funcționează doar dacă utilizatorul este verificat

Concluzii

Toate aceste riscuri identificate au legătură numai cu sistemul de autentificare și autorizare, și de modul în care este scris codul aplicației. Pe lânge aceste probleme, mai pot fi identificate riscuri de securitate legate de sistemul de fișiere al serverului web, riscuri legate de XSS „cross-site scripting”, probleme de phishing provenite din lipsa de securitate la CSRF.

Acest proiect dezvăluie cât de importantă este securizarea aplicațiilor web și nu numai, și în câte moduri un atacator poate reuși să obțină acces la date digitale nesecurizate corespunzător.

Link-uri

Git Repository:

Branch Main: https://github.com/RoTak00/sec-sof-proiect/tree/main
Branch "Fixed": https://github.com/RoTak00/sec-sof-proiect/tree/fixed

Video-uri:

Stocarea parolelor: https://youtu.be/tZgFPM4lWw8
Cerințe de securitate ale parolelor: https://youtu.be/XETg0PsdIAg
Resetarea Parolelor: https://youtu.be/m7GA_3b9ApU
Operații Neautorizate: https://youtu.be/-eoDtqIaixY
Brute-forcing: https://youtu.be/LYUeHFB9nGM
Enumerare de conturi: https://youtu.be/sYhjICratxY